Chapter 1/Migrating-To-Docker-Compose

Migrating to Docker Compose

Even with a simple image, we've already been dealing with plenty of command line options in both building, pushing and running the image.

Next we will switch to a tool called Docker Compose ↗ to manage these. Docker Compose used to be a separate tool but now it is integrated into Docker and can be used like the rest of the Docker commands.

> Docker Compose documentation states that using .yaml extension is preferred over .yml even though both are acceptable. You will most likely see both extension types when reading tutorials, documentations and project examples. In this course we will have docker-compose.yaml in use, but you can choose the newest standard for naming, which is compose.yaml.

Docker Compose is designed to simplify running multi-container applications using a single command `docker compose [-f <arg>...] [options] [COMMAND] [ARGS...]`.

Assume that we are in the folder where we have our Dockerfile with the following content:

```
FROM debian:10.10

WORKDIR /app/

RUN apt-get update && apt-get install -y curl python3 ffmpeg
RUN curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp -o /usr/local/bin/yt-dlp
RUN chmod a+rx /usr/local/bin/yt-dlp

ENTRYPOINT ["/usr/local/bin/yt-dlp"]
```

Let us now create a file called `docker-compose.yaml`:

```
services:
  yt-dlp-ubuntu:
    image: <username>/<repositoryname>
    build:
```

File defines now one service that is given name `yt-dlp-ubuntu`.

The value of the key `build` can be a file system path (in the example it is the current directory `.`) or an object with keys `context` and `dockerfile`, see the documentation ↗ for more

Now we can build and push with just these commands:

```
$ docker compose build
$ docker compose push
```

Volumes in Docker Compose

To run the image as we did previously, we will need to add the volume bind mounts. Volumes in Docker Compose are defined with the familiar syntax `location-in-host:location-in-container`. Compose can work without an absolute path:

```
services:
  yt-dlp-ubuntu:
    image: <username>/<repositoryname>
    build:
    volumes:
      - ./dyp2:/
    container_name: yt-dlp
```

We can also give the container a name it will use when running with container_name. The service name can be used to run the container:

```
$ docker compose run yt-dlp-ubuntu https://www.youtube.com/watch?v=dQw4w9WgXcQ
```

Using a ready image with Docker Compose

In the previous example, we used the key `build` to tell how the `image` defined in the compose file is built if the command `docker compose build` is used. It is pretty common that we use some readily built images, and in that case, the key `build` is not needed. We could e.g., use the following docker-compose.yaml file to define two containers, one based on image nginx:1.27 and the other based on postgres:17:

```
services:
  nginx:
    image: nginx:1.27
  database:
    image: postgres:17
```

Both of the containers can now be started with command `docker compose up`:

```
nginx-1   | 2025/10/19 13:05:20 [notice] 1#1: start worker process 39
nginx-1   | 2025/10/19 13:05:20 [notice] 1#1: start worker process 40
nginx-1   | 2025/10/19 13:05:20 [notice] 1#1: start worker process 41
nginx-1   | 2025/10/19 13:05:20 [notice] 1#1: start worker process 42
database-1 | Error: Database is uninitialized and superuser password is not specified.
database-1 |        You must specify POSTGRES_PASSWORD to a non-empty value for the
database-1 |        superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run*.
database-1 |
database-1 |        You may also use POSTGRES_HOST_AUTH_METHOD=trust to allow all
database-1 |        connections without a password. This is *not* recommended.
database-1 |
database-1 |        See PostgreSQL documentation about "trust":
database-1 |        https://www.postgresql.org/docs/current/auth-trust.html
database-1 | exited with code 1
```

The nginx web server starts up but as we can see from the log output the Postgres database doesn't start due to some missing configurations. However, we shall not care about that for now.

Key Commands in Docker Compose

Before going to exercises, let us go through the commands that you need most often:

To start all the services defined in the `docker-compose.yaml` file:

```
$ docker compose up
```

To stop and remove the running services:

```
$ docker compose down
```

If you want to monitor the output of the running containers and debug issues, we can view the logs with:

```
$ docker compose logs
```

To lists all the services along with their current status:

```
$ docker compose ps
```

Full list of Compose CLI Commands can be found here.

Exercise 2.1. Simple service writing to log — max. score 10 h, 0 h

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

Let us now leverage the Docker Compose with the simple web service that we used in the Exercise 1.3

Without a command `devopsdockuhub/simple-web-service` will create logs into the `/usr/src/app/test.log`.

Create a docker-compose yaml file that starts `devopsdockuhub/simple-web-service` and saves the logs into your filesystem.

Submit the docker-compose.yaml, and make sure that it works simply by running `docker compose up` if the log file exists.

```
services:
  webs:
    image devopsdockuhub/simple-web-service
    volumes:
      - /logs/test.log:/usr/src/app/test.log
    container_name webs
```

Word count: 9

Web services in Docker Compose

Compose is really meant for running web services, so let us get really started

https://github.com/jwilder/whoami ↗ is a simple service that prints the current container id (hostname).

```
$ docker container run -d -p 8000:8000 jwilder/whoami
f9ca6b67c1c6ccb0d8d8b0c3373d7c8c8d8c8a8f7c2c8b8b9ca3a4a2b9
```

Navigate with a browser or curl to localhost:8000, they both will answer with the id.

Take down the container so that it's not blocking port 8000.

```
$ docker container stop f9ca6b67c1c
$ docker container rm f9ca6b67c1c
```

Let's create a new folder and a Docker Compose file `whoami/docker-compose.yaml`, from the command line options.

```
services:
  whoami:
    image: jwilder/whoami
    ports:
      - 8000:8000
```

Test it:

```
$ docker compose up -d
$ curl localhost:8000
```

As you can see, the flag `-d` started the container in detached mode. When you are ready, you can stop and remove the container with `docker compose down`.

Environment variables can also be given to the containers in Docker Compose as follows:

```
services:
  backend:
    image:
    environment:
      - VARIABLE=VALUE
      - VARIABLE2=VALUE2
```

Note that there are also other ↗, perhaps more elegant ways to define the environment variables in Docker compose.

Exercise 2.2. Simple service with browser — max. score 10 h, 0 h

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

The familiar image `devopsdockuhub/simple-web-service` can be used to start a web service, see the exercise 1.10

Create a docker-compose yaml, and use it to start the service so that you can see it with your browser.

- Read about how to add the `command` to docker-compose yaml from the documentation ↗

Submit the docker-compose.yaml, and make sure that it works simply by running `docker compose up`

```
services:
  webs:
    image devopsdockuhub/simple-web-service
    ports:
      - 8080:8080
    command [server1]
    container_name webs
```

Word count: 10

Exercise 2.3. Project with compose (mandatory exercise) — max. score 10 h, 0 h

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

As we see previously, starting an application with two programs was not trivial and the commands got a bit long.

In the previous part we created Dockerfiles for both frontend ↗ and backend ↗ of the example application. Next, simplify the usage into one docker-compose.yaml.

Configure the backend and front end to work in Docker Compose, as in Exercise 1.9 ↗. If the button for Exercise 1.9 works, your setup is correct.

You may assume that the images are readily built and available locally or in Docker Hub.

Submit the docker-compose.yaml

```
services:
  frontend
    image frontend
    ports
      - 5000:5000
    command ./naersk ./naersk-frontend
    ports
      - 3000:3000
    container_name frontend
  backend
    image backend
    ports
    command ./naersk ./naersk-backend
    ports
      - 8080:80
    container_name backend
```

Word count: 23

Chapter front page

You've reached the end of this topic.

Proceed to the next topic

Next topic
Docker networking

About

The University of Helsinki MOOC Center makes high-quality online education possible by developing and researching educational software and online learning materials. Teachers both within and without the University of Helsinki rely on our tools to create impactful teaching materials. Our popular Massive Open Online Courses (MOOCs) have been available through MOOC.fi since 2012.

This website is powered by an open source software developed by the University of Helsinki MOOC Center. Star the project on GitHub. Project GitHub.

MOOC.FI

Privacy
Accessibility statement

GIVE FEEDBACK